嵌入式作業系統
--LAB02

姓名:邱榮誠

學號:314512065

系級:電控碩 1

日期:2025/09/26

Q1: Shrink the size of your kernel image

1) **目的**:由於預設的 RPi OS image 通常包含許多實際上用不到的模組，透過刪除這些多餘項目，不僅能釋放更多空間供應用程式與資料使用，也能使系統檔案更精簡，降低 SD 卡的讀寫負擔並縮短開機時間。若在實務應用中僅需滿足特定需求，此種精簡方式更能有效降低成本，提升系統效能。
2) **移除功能**:
   - File systems 內的 F2FS filesystem support(0.7MB)
   - General architecture-dependent options 內的 Stack Protector buffer overflow detection(0.4MB)
3) **刪除理由**:
   - 因 F2FS 是專門為 NAND/eMMC/SSD 優化的檔案系統，但是這次 LAB2 的 boot partition 使用 fat32，root partition 使用 ext4，編譯過程中沒有掛載或格式化成 F2FS 的需求，因此將其刪除。
   - 因 Stack Protector 是一種安全機制，編譯器會在函式中插入額外的檢查碼，用來偵測與防止 stack-based buffer overflow，但是在這次 LAB2 中，沒有系統安全性的考量，因此將其刪除。
4) **如何移除**:
   - 建設好環境
   - git clone --depth=1 --branch rpi-6.1.y https://github.com/raspberrypi/linux: 得到 kernel source
   - cd linux:進入 linux 資料夾
   - KERNEL=kernel8:指定 KERNEL 檔名為 kernel8
   - sudo apt install libncurses5-dev:安裝 ncurses 函式庫的開發套件，後續 menuconfig 須利用到。
   - make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- menuconfig: 顯示選單，可在選單內移除 kernel 的功能。
5) **kernel 移除前後功能比較**:

Table1. kernel 移除前後功能比較

	kernel 移除功能前	kernel 移除功能後
F2FS filesystem support	可支援 NAND/eMMC/SSD 的最佳化檔案系統，需對應 F2FS 格式分割區。	無法掛載 F2FS。
Stack Protector buffer overflow detection	可防止 stack-based buffer overflow，提高安全性。	編譯與執行稍快、binary 變小，但失去安全檢查。

6) **Image size reduction result**:

23.6 MB ->22.6MB

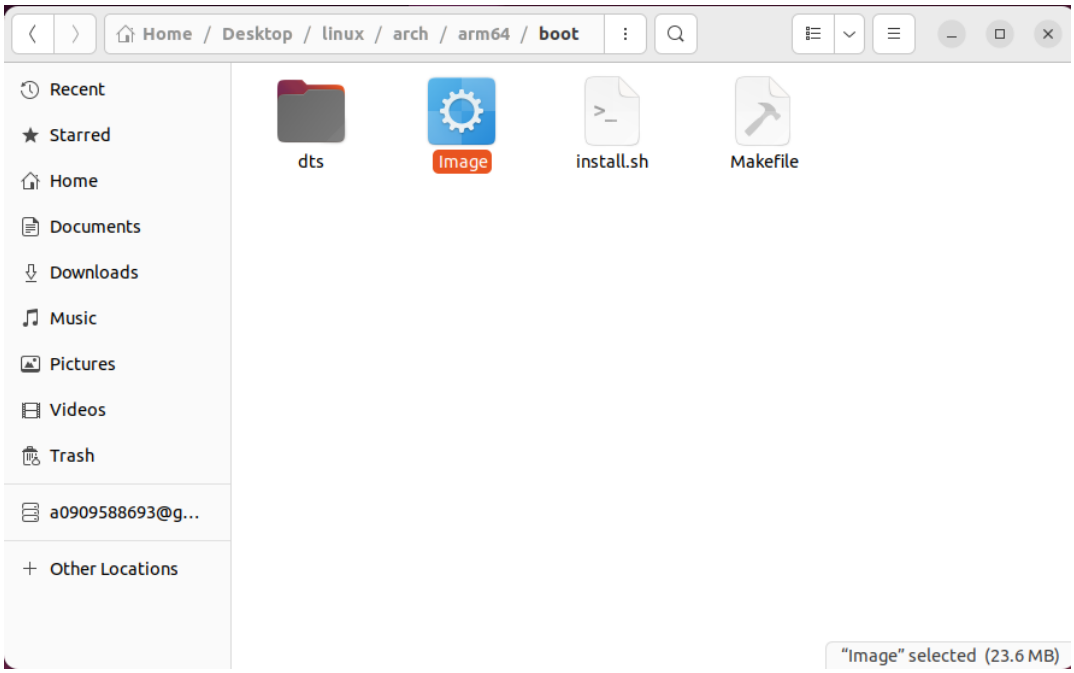

Figure1.Original image size

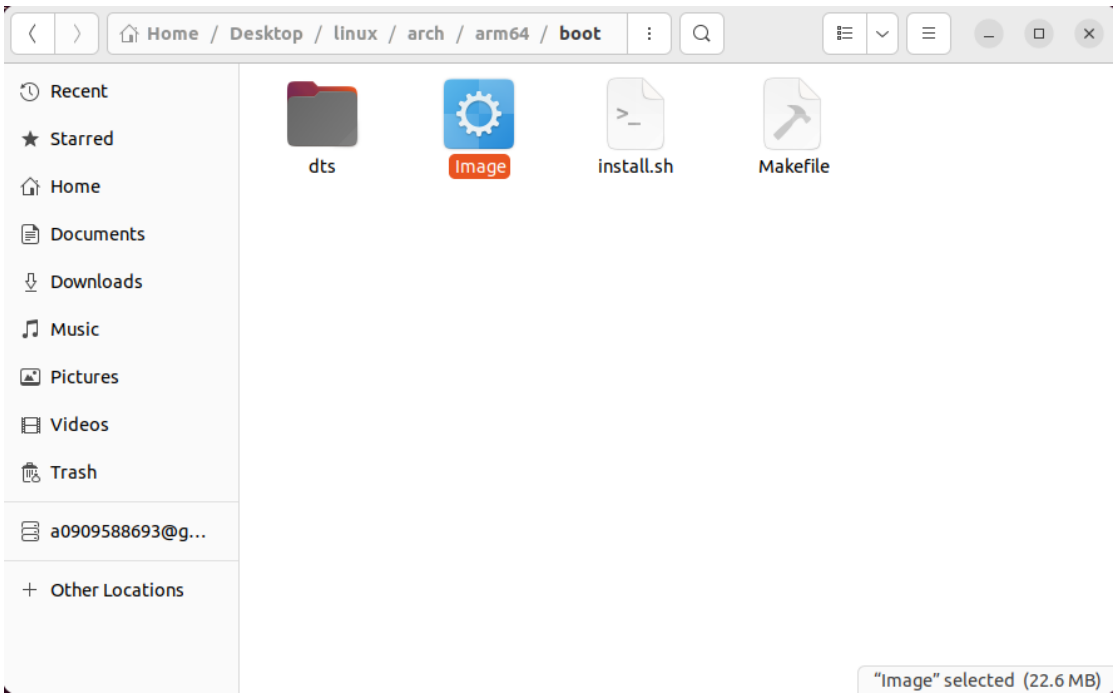

Figure2.Reducted image size

Q2: Benchmark your kernel, revise it, and improve its performance. Rerun the benchmark to prove your performance.

1) **下載評測kernel的套件**: 採用不下載kernel套件的方式

2) **評測結果**:

Table2.評測結果

	移除功能前	移除功能後
記憶體測試	211 MB/s	137 MB/s
程式執行時間測試	665 MB/s（1.59秒完成1GB）	1.0 GB/s（1.03秒完成1GB）

```
pi@raspberrypi:~ $ dd if=/dev/zero of=/dev/shm/test bs=1M count=500
dd: error writing '/dev/shm/test': No space left on device
455+0 records in
454+0 records out
476737536 bytes (477 MB, 455 MiB) copied, 2.26245 s, 211 MB/s
```

Figure3. 記憶體測試(移除功能前)

```
pi@raspberrypi:~ $ dd if=/dev/zero of=/dev/shm/test bs=1M count=500
dd: error writing '/dev/shm/test': No space left on device
455+0 records in
454+0 records out
476311552 bytes (476 MB, 454 MiB) copied, 3.47529 s, 137 MB/s
```

Figure4. 記憶體測試(移除功能後)

```
pi@raspberrypi:~ $ time dd if=/dev/zero of=/dev/null bs=1M count=1000
1000+0 records in
1000+0 records out
1048576000 bytes (1.0 GB, 1000 MiB) copied, 1.57693 s, 665 MB/s

real    0m1.591s
user    0m0.001s
sys     0m0.872s
```

Figure5. 程式執行時間測試(移除功能前)

```
pi@raspberrypi:~ $ time dd if=/dev/zero of=/dev/null bs=1M count=1000
1000+0 records in
1000+0 records out
1048576000 bytes (1.0 GB, 1000 MiB) copied, 1.00778 s, 1.0 GB/s

real    0m1.034s
user    0m0.005s
sys     0m0.700s
```

Figure6. 程式執行時間測試(移除功能後)

3) **說明**:可以看到程式執行時間測試，移除功能後效能更好。因Stack Protector buffer overflow detection每次進入特定函式時，編譯器會額外插入一段程式碼來放置 canary值，函式返回前還要驗證canary是否被修改。這增加了額外指令（尤其是函式呼叫頻繁的小函式），但是移除功能後，少了這些額外檢查，CPU pipeline可以更快執行。相較之下，記憶體測試效能卻下降，可能是因為F2FS filesystem support移除後，失去針對快閃儲存的記憶體最佳化，而導致的現象。

Q3: Patch your kernel to support real-time tasks.

1) 下載哪一版本的補丁:4.19.127

2) 如何下載

- git clone --depth=1 --branch rpi-4.19.y https://github.com/raspberrypi/linux: 得到 kernel source
- 找到 patch-4.19.127-rt54.patch.gz 下載
- gunzip patch-4.19.127-rt54.patch.gz 解壓縮
- cat patch-4.19.127-rt54.patch | patch -p1
- 執行 make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- bcm2711_defconfig
- 執行 make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- menuconfig
- 進入 General setup 找到 Preemption Model(RT)選擇(RT)
- 執行 make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- Image modules dtbs
- 將編譯結果 cp 到 SD 上
- 進入 RPI，指令:uname -r 查看是否 patch 成功

3) 下載的結果

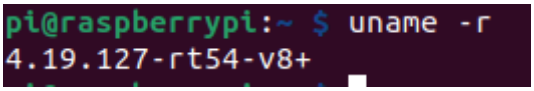

Figure6. Patch後的結果

Q4: Experience and suggestions

心得:

這次的Lab是我第一次嘗試使用開發板來實作。實際碰過才知道開發板的問題真的好多，上一秒連得到下一秒就連不到，每次處理這些問題怒氣值就會莫名飆升，明明乖乖也沒少買啊。本次實驗花了許久的時間，不斷try and error目標在最小kernel image size下仍維持rpi開機功能，但因為電腦核心數不夠，導致編譯時間漫長，所以最後在茫茫menuconfig中只找到兩個符合目標的功能將其刪除，雖然最終的成果不是很理想，但至少讓我對嘗試過的功能有更深入的了解(例如:在Power management options中有Device power management core functionality這個功能，它負責整個裝置驅動初始化，所以將其刪除會使rpi開不了機)。經由上述，我想這種從實驗中自學的方式，應該是老師設計這個LAB背後隱藏的意涵吧。最後也謝謝助教們在暑假先幫我們課前加強，讓我們能更快進入狀況，期許自己之後的LAB完成度能更上一層樓!

建議:

能理解老師設計這個LAB的用意，是要讓我們體會kernel功能取捨與裁剪的觀念。但相較後續的LAB3、4、5…，在實作上就少了一些成就感。以LAB3為例，當七段顯示器真的跑出學號的那一刻，會有一種自己完成了功能的滿足感，也因此更有動力去挑戰後面的實驗，但LAB2的成果主要只是看到image

size的數字變大或變小，我覺得偏單調。如果能在設計上加入一些更具創意或能看得見成果的元素，讓我們在學習kernel裁剪的同時，也能獲得更多實作上的成就感，那可能更好玩一些。(希望不要在我們當助教的時候創新)